ASSIGNMENT-17.3

NAME: G. MANI SHARAN
ROLL.NO: 2303A52148
BATCH: 41

Task 1 – Employee Dataset Cleaning

Prompt:

Clean an employee dataset:

Instructions:
- Handle missing values in salary (median), designation (mode), and experience (mean)
- Remove duplicate records based on employee_id
- Convert designation text to lowercase
- Encode categorical columns: designation and employment_type
- Output the cleaned dataset with no missing values and encoded features

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder

# ---- Sample Dataset ----
np.random.seed(42)
df = pd.DataFrame({
    "employee_id": np.random.randint(100, 200, 50),
    "name": [f"Emp{i}" for i in range(50)],
    "salary": np.random.choice([30000, 40000, 50000, 60000, np.nan], 50),
```

```
    "designation": np.random.choice(["Manager", "Developer",
"Analyst", None], 50),
    "experience": np.random.choice([1, 2, 3, 4, 5, np.nan], 50),
    "employment_type": np.random.choice(["Full-time", "Part-time",
"Contract"], 50)
})

# ---- 1. Handle Missing Values ----
df['salary'] = df['salary'].fillna(df['salary'].median())
df['designation'] =
df['designation'].fillna(df['designation'].mode()[0])
df['experience'] = df['experience'].fillna(df['experience'].mean())

# ---- 2. Remove Duplicate Employee IDs ----
df = df.drop_duplicates(subset='employee_id')

# ---- 3. Standardize Text Values ----
df['designation'] = df['designation'].str.lower()

# ---- 4. Encode Categorical Features ----
le1 = LabelEncoder()
le2 = LabelEncoder()

df['designation'] = le1.fit_transform(df['designation'])
df['employment_type'] = le2.fit_transform(df['employment_type'])

# ---- Output ----
print("Cleaned Dataset:")
print(df.head())

print("\nShape:", df.shape)
print("\nMissing Values:\n", df.isnull().sum())
```

Output:

```
Cleaned Dataset:
   employee_id  name   salary  designation  experience  employment_type
0          151  Emp0  40000.0            0         2.0                0
1          192  Emp1  60000.0            0         2.0                1
2          114  Emp2  30000.0            0         4.0                2
3          171  Emp3  60000.0            0         5.0                1
4          160  Emp4  40000.0            0         3.0                1

Shape: (37, 6)

Missing Values:
 employee_id        0
name                0
salary              0
designation         0
experience          0
employment_type     0
dtype: int64
```

Justification:

- Missing Values Handling*

- Salary → Median: Salary distributions are often skewed; median is robust to outliers.
- Designation → Mode: Most frequent job title maintains categorical consistency.
- Experience → Mean: Numerical field; mean preserves overall distribution.

- Duplicate Removal

- Removing duplicate employee_id ensures *data integrity* and avoids biased analytics.

- Text Standardization

- Converting designations to lowercase eliminates *case-sensitive inconsistencies* (e.g., “Manager” vs “manager”).

- Categorical Encoding

- Machine learning models require numerical inputs; encoding converts text into usable numeric form.

Task 2 – Student Performance Data Preprocessing

Task:

Preprocess student performance data using AI assistance.

Prompt:

Preprocess a student performance dataset.

Instructions:
- Convert exam_date to datetime format
- Extract new features: month, day, semester (1 for Jan–Jun, 2 for Jul–Dec)
- Normalize total_marks using MinMax scaling
- Detect and remove outliers using IQR method
- Output the transformed dataset with new features and normalized marks

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# ---- Sample Dataset ----
np.random.seed(0)
dates = pd.date_range(start="2024-01-01", periods=60)

df = pd.DataFrame({
    "student_id": range(1, 61),
    "exam_date": np.random.choice(dates, 60),
    "total_marks": np.random.randint(200, 1000, 60)
})

# Add some outliers
```

```
df.loc[5, 'total_marks'] = 1500
df.loc[10, 'total_marks'] = 50

# ---- 1. Convert exam_date to datetime ----
df['exam_date'] = pd.to_datetime(df['exam_date'])

# ---- 2. Extract Features ----
df['month'] = df['exam_date'].dt.month
df['day'] = df['exam_date'].dt.day
df['semester'] = df['month'].apply(lambda x: 1 if x <= 6 else 2)

# ---- 3. Normalize total_marks ----
scaler = MinMaxScaler()
df['total_marks'] = scaler.fit_transform(df[['total_marks']])

# ---- 4. Detect & Remove Outliers (IQR) ----
Q1 = df['total_marks'].quantile(0.25)
Q3 = df['total_marks'].quantile(0.75)
IQR = Q3 - Q1

df = df[(df['total_marks'] >= Q1 - 1.5 * IQR) &
        (df['total_marks'] <= Q3 + 1.5 * IQR)]

# ---- Output ----
print("Processed Dataset:")
print(df.head())

print("\nShape:", df.shape)
print("\nColumns:", df.columns)
```

Output:

```
Processed Dataset:
   student_id  exam_date  total_marks  month  day  semester
0           1 2024-02-14     0.229655      2   14         1
1           2 2024-02-17     0.122759      2   17         1
2           3 2024-02-23     0.191724      2   23         1
3           4 2024-01-01     0.191724      1    1         1
4           5 2024-01-04     0.140000      1    4         1

Shape: (59, 6)

Columns: Index(['student_id', 'exam_date', 'total_marks', 'month', 'day', 'semester'], dtype='object')
```

Justification:

- *Datetime Conversion*

- Enables *time-based feature extraction* and chronological analysis.

- *Feature Extraction (month, day, semester)*

- Captures *academic patterns and seasonality* (e.g., exam trends across semesters).

- *Normalization (MinMax Scaling)*

- Scales marks to a uniform range (0–1), preventing *feature dominance* in ML models.

- *Outlier Removal (IQR)*

- Eliminates extreme values that can distort model training and statistical analysis.

Task 3 – Student Health Data Preparation

Task:

Clean and preprocess student health records using AI scripts.

Prompt:

Prepare a student health dataset for machine learning.

Instructions:
- Handle missing values in height, weight, and BMI using mean
- Encode categorical features: medical_condition and fitness_status

- Apply standard scaling to numerical features (height, weight, BMI)
- Split dataset into training and testing sets (80-20)
- Output processed dataset ready for modeling

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

# ---- Sample Dataset ----
np.random.seed(1)

df = pd.DataFrame({
    "student_id": range(1, 51),
    "height": np.random.choice([150, 160, 170, 180, np.nan], 50),
    "weight": np.random.choice([50, 60, 70, 80, np.nan], 50),
    "BMI": np.random.choice([18, 22, 25, 30, np.nan], 50),
    "medical_condition": np.random.choice(["None", "Asthma",
"Diabetes"], 50),
    "fitness_status": np.random.choice(["Fit", "Unfit"], 50)
})

# ---- 1. Handle Missing Values ----
df['height'] = df['height'].fillna(df['height'].mean())
df['weight'] = df['weight'].fillna(df['weight'].mean())
df['BMI'] = df['BMI'].fillna(df['BMI'].mean())

# ---- 2. Encode Categorical Features ----
le1 = LabelEncoder()
le2 = LabelEncoder()

df['medical_condition'] = le1.fit_transform(df['medical_condition'])
df['fitness_status'] = le2.fit_transform(df['fitness_status'])

# ---- 3. Scale Numerical Features ----
scaler = StandardScaler()
```

```
df[['height', 'weight', 'BMI']] = scaler.fit_transform(df[['height', 'weight',
'BMI']])

# ---- 4. Train-Test Split ----
X = df.drop('fitness_status', axis=1)
y = df['fitness_status']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# ---- Output ----
print("Processed Dataset Sample:")
print(df.head())

print("\nTrain Shape:", X_train.shape)
print("Test Shape:", X_test.shape)
```

Output:

```
Processed Dataset Sample:
   student_id    height        weight       BMI  medical_condition  \
0           1  1.791095 -4.125904e-01 -0.213982                  0
1           2  0.000000  5.930988e-01 -1.394570                  0
2           3 -1.399293 -1.418280e+00  0.671460                  2
3           4 -0.335830  1.429170e-15  0.000000                  0
4           5  1.791095 -4.125904e-01  0.000000                  2

   fitness_status
0               0
1               0
2               0
3               1
4               0

Train Shape: (40, 5)
Test Shape: (10, 5)
```

Justification:

- *Missing Value Imputation (Mean)*

- Maintains dataset size while preserving average distribution of health metrics.

- *Categorical Encoding*

- Converts health conditions and fitness status into numerical form for predictive modeling.

- *Standard Scaling*

- Centers data (mean=0, std=1), essential for algorithms sensitive to feature magnitude (e.g., SVM, KNN).

- *Train-Test Split*

- Separates data to evaluate model performance on unseen data, ensuring *generalization*

Task 4: Real-Time Application: Ride-Sharing Data Cleaning

Scenario:

A ride-sharing company has trip data with errors.

Prompt:

Clean a ride-sharing dataset.

Instructions:
- Standardize pickup and drop locations to lowercase
- Fill missing fare values using median
- Remove duplicate records based on ride_id
- Normalize driver_rating using MinMax scaling
- Generate summary of cleaning steps and final dataset

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# ---- Sample Dataset ----
```

```
np.random.seed(2)

locations = ["Chennai", "Adyar", "T Nagar", "Velachery", "Tambaram"]

df = pd.DataFrame({
    "ride_id": np.random.randint(1, 40, 60),
    "pickup": np.random.choice(locations, 60),
    "drop": np.random.choice(locations, 60),
    "fare": np.random.choice([100, 150, 200, 250, np.nan], 60),
    "driver_rating": np.random.uniform(3.0, 5.0, 60)
})

# Introduce inconsistencies
df.loc[0, 'pickup'] = "chennai"
df.loc[1, 'drop'] = "adyar"

# ---- 1. Standardize Locations ----
df['pickup'] = df['pickup'].str.lower()
df['drop'] = df['drop'].str.lower()

# ---- 2. Handle Missing Fare ----
df['fare'] = df['fare'].fillna(df['fare'].median())

# ---- 3. Remove Duplicate Rides ----
before_count = len(df)
df = df.drop_duplicates(subset='ride_id')
after_count = len(df)

# ---- 4. Normalize Driver Ratings ----
scaler = MinMaxScaler()
df['driver_rating'] = scaler.fit_transform(df[['driver_rating']])

# ---- 5. Summary ----
print("CLEANED DATASET SAMPLE:")
print(df.head())

print("\nSUMMARY:")
```

```
print("Records Before Cleaning:", before_count)
print("Records After Removing Duplicates:", after_count)
print("Missing Values:\n", df.isnull().sum())
```

Output:

```
••• CLEANED DATASET SAMPLE:
   ride_id     pickup       drop   fare  driver_rating
0       16    chennai    chennai  200.0       0.976876
1        9    t nagar      adyar  100.0       0.787486
2       23   tambaram    chennai  100.0       0.423726
3       19    t nagar  velachery  250.0       0.341723
4       12  velachery      adyar  150.0       0.539948

SUMMARY:
Records Before Cleaning: 60
Records After Removing Duplicates: 28
Missing Values:
 ride_id          0
pickup            0
drop              0
fare              0
driver_rating     0
dtype: int64
```

Justification:

- *Location Standardization*

- Ensures consistent grouping and analysis (e.g., “Chennai” vs “chennai”).

- *Missing Fare Imputation (Median)*

- Median prevents distortion caused by unusually high/low fares.

- *Duplicate Removal*

- Avoids repeated ride entries, ensuring accurate operational metrics.

- *Rating Normalization*

- Scales ratings for fair comparison and compatibility with ML models.

- *Summary Generation*

- Provides transparency and validates improvements in data quality